Axios Deep Dive: Professional Notes for Beginners

This document outlines key concepts and advanced usage patterns for the Axios HTTP client library, often preferred in professional web development for its robust feature set and ease of use. This knowledge is essential for any modern frontend or backend developer working with API communication.

1. General Axios Information

Axios is a promise-based HTTP client for the browser and Node.js. It simplifies API interactions and is preferred by many developers due to its intuitive API and powerful features.

Feature	Description
Promise-Based	Utilizes Promises, making asynchronous code cleaner.
Browser/Node.js	Works in both environments without configuration changes.
Interceptors	Allows for intercepting requests and responses (more on this later).
Automatic Transformations	Automatically serializes request data and parses response data.

2. Custom Instance Methods

Axios allows for creating custom instances with pre-defined configurations, which promotes code reuse and separation of concerns (e.g., separating an authentication API client from a product data API client).

The methods on a custom instance are flexible, supporting various signature patterns:

- **Custom Instance with Data (POST/PUT/PATCH):**
  `customInstance.method(url, body, config)`
  - *Example:* `customApi.post('/users', { name: 'John' }, { headers: { 'X-Custom': 'Value' } })`
- **Custom Instance without Data (GET/DELETE/HEAD):**
  `customInstance.get(url, config)`
  - *Example:* `customApi.get('/users/123', { timeout: 5000 })`

3. Why Axios is Preferred

Axios is the preferred choice over the native `fetch` API for several key reasons, simplifying development workflow significantly:

- **Automatic Header Setting:** Axios automatically sets the `Content-Type` header based on the request data (e.g., to `application/json` for JavaScript objects).
- **Automatic Data Serialization:** It automatically converts JavaScript objects to JSON for outgoing requests.
- **Automatic Response Deserialization:** It automatically parses JSON response data into JavaScript objects, eliminating the need for an explicit `.json()` call.

4. Understanding Response Objects

A custom Axios instance, upon receiving a response from the server, returns a structure that includes the request configuration, the response data, and the error object (if a server error occurs).

- **`config` Object:**
  This object contains the original request information, such as the URL, method, headers, and any custom configuration passed. It is always accessible.
- **`response` Object:**
  This object contains the response generated by the server. **Crucially, the actual**

data payload from the server is nested inside the `data` key.// Example of destructuring the data

```
const { data } = await customInstance.get('/products');

// 'data' now holds the server's payload
```

- **`error` Object:**
  This object is returned when the server responds with a status code outside the 2xx range (e.g., 400, 401, 500). It contains both the request configuration and the response details. This allows for advanced error handling.
  - You can access the original request details: `error.config`
  - You can access the server's response (for non-2xx statuses): `error.response`

Important Note: The `response` and `error` objects are only generated and accessible **when a response or error is provided BY THE SERVER ONLY**. The `config` object, however, is always accessible (e.g., in request interceptors).

5. Interceptors and Refresh Tokens

Axios interceptors are powerful tools for managing side effects globally, such as automatically handling authentication errors like expired refresh tokens.

Interceptors can be set up for both requests and responses.

- **Use Case: Handling `401 Unauthorized` for Refresh Tokens**
  a. A request fails with a `401` status.
  b. The **response interceptor** catches the error.
  c. It checks the status: `error.response.status` is `401`.
  d. The interceptor then calls the refresh token endpoint to get a new token.
  e. Finally, it retries the original request by returning the custom instance with the failed request's configuration: `return axiosInstace(error.config)`. This passes the whole original request object back to the instance, which executes the request again, this time with the new token.

6. Error Handling in Interceptors and React Query

When an interceptor catches an error (such as a 401), it must ensure the downstream promise chain is correctly informed about the failure, unless the error is successfully handled (like in the refresh token example).

- **General Rule:** In an error interceptor, you **must reject a promise** to propagate the error down the chain:return Promise.reject(error);
- **React Query Context:** When using Axios as a `queryFn` in React Query, the function essentially returns a promise. For React Query to correctly flag the query as an error, the `queryFn` (and thus the underlying Axios call) **must throw an error or reject the promise**.

7. Progress Tracking

Axios provides built-in properties in the request configuration to track upload and download progress, allowing developers to implement professional progress bars in the UI.

Property	Direction	Description	Location
`onUploadProgress`	Outgoing	Callback function to track the progress of data being sent to the server (e.g., file uploads).	`config` object
`onDownloadProgress`	Incoming	Callback function to track the progress of data being received from the server (e.g., large data downloads).	`config` object

8. Request Cancellation

Axios offers functionality to cancel requests before they complete. While useful for manual control, it's often unnecessary when integrated with state management libraries like React Query.

- **React Query:** When a component unmounts or a query becomes inactive, React Query's built-in memory management and lifecycle handling implicitly manage the cancellation of active queries, making explicit Axios cancellation less critical.